

Table des matières

1	Introduction	2
2	Définition du problème	2
3	Structure de données utilisée	2
4	Analyse des Algorithmes Envisagés	2
4.1	Algorithme 1 : Distance de sécurité avec l'ennemi	2
4.2	Algorithme 2 : Position défensive dans un coin	2
4.3	Algorithme 3 : Convergence collective vers une cible (retenu)	2
4.4	Algorithme 4 : Séparation des rôles détection/combat (retenu)	3
5	Critique des Algorithmes de la Littérature	3
5.1	Intelligence en essaim	3
5.2	ACO distribué et modèle d'acteur	3
5.3	Programmation orientée agents	3
5.4	Systèmes multi-agents et représentation des connaissances	3
6	Présentation de l'Algorithme Implémenté	3
6.1	Organisation de l'équipe	3
6.2	Partage de la position ennemie	4
6.3	Réception des messages	4
7	Tests de Performance	4
7.1	Méthode d'obtention des données de test	4
7.2	Résultats	5
7.3	Ce qui pourrait être amélioré	5
8	Conclusion	6

1 Introduction

Ce rapport parle du modèle d'acteur appliqué aux systèmes multi-agents (SMA), avec le simulateur Simovies. Dans un SMA, plusieurs entités autonomes agissent en parallèle : c'est exactement ce que permet le modèle d'acteur, où chaque entité agit seule, garde ses informations pour elle, et communique avec les autres uniquement en envoyant des messages. Pour illustrer ces concepts, on commence par expliquer les différentes stratégies qu'on a envisagées pour nos robots, puis on détaille celle qu'on a retenue et comment on l'a implémentée, avant de terminer par les résultats obtenus aux tests de performances.

2 Définition du problème

Deux équipes de robots s'affrontent sur un terrain, et chaque équipe doit immobiliser le plus de robots adverses possible. Chaque robot ne voit que ce que son radar détecte autour de lui. Le défi est donc de coordonner une équipe de robots qui n'ont qu'une vision partielle de la situation.

3 Structure de données utilisée

Dans notre implémentation, chaque robot utilise `enum State` définit l'état courant du robot : `MOVING`, `CHASING`, `FIRING`, `REPOSITIONING`. `enum Identity` définit l'identité du robot : `ROCKY`, `MARIO`, `GEORGE`. La classe `Position` stocke les coordonnées (`x`, `y`) du robot sur le terrain. Chez les `MainBots` uniquement, `enemyPosition` est une instance de `Position` qui contient la dernière position ennemie connue. Enfin, l'`ArrayList<String>` `messages` contient les messages reçus des coéquipiers : soit des coordonnées (`x,y`) pour signaler la position d'un ennemi, soit `"RESET"` pour indiquer qu'une cible est détruite.

4 Analyse des Algorithmes Envisagés

4.1 Algorithme 1 : Distance de sécurité avec l'ennemi

La première idée était de faire maintenir à chaque robot une distance fixe par rapport à l'ennemi, en reculant si celui-ci se rapproche trop et en avançant s'il s'éloigne. Cela réduit les risques d'être touché mais dès qu'un robot est encerclé par plusieurs ennemis, il ne sait plus dans quelle direction se déplacer.

4.2 Algorithme 2 : Position défensive dans un coin

Cette approche consiste à installer l'équipe dans un coin de la carte au hasard, avec les `SecondaryBots` positionnés en avant pour protéger les `MainBots` qui sont alors difficiles à attaquer. En revanche, le trajet jusqu'au coin choisi peut exposer les robots à des attaques avant qu'ils ne soient en position.

4.3 Algorithme 3 : Convergence collective vers une cible (retenu)

Dès qu'un robot voit un ennemi, il partage sa position avec toute l'équipe via `broadcast()`, et tous les robots convergent vers cette cible. Cela permet d'éliminer l'ennemi rapidement mais si un autre ennemi est détecté avant que le premier ne soit détruit, les robots changent de cible, ce qui peut rendre l'équipe inefficace.

4.4 Algorithme 4 : Séparation des rôles détection/combat (retenu)

Les SecondaryBots explorent la carte et signalent les ennemis, tandis que les MainBots se chargent de les poursuivre et de tirer. Cela permet d'avoir une vision globale de la carte. L'inconvénient est que les SecondaryBots ne participent jamais au combat, ce qui peut laisser les MainBots sans soutien.

5 Critique des Algorithmes de la Littérature

5.1 Intelligence en essaim

Les algorithmes d'essaim [1], comme ACO (colonie de fourmis) ou PSO (essaim de particules), reposent sur des règles locales très basiques appliquées par chaque agent qui peuvent produire un comportement collectif efficace. C'est l'idée qu'on cherche à reproduire dans Simovies. Le problème, c'est que ces algorithmes ont besoin de plusieurs itérations pour converger vers une bonne solution. Or dans Simovies, chaque robot doit décider en un seul step : on ne laisse pas de temps à l'algorithme pour "réfléchir" avant d'agir.

5.2 ACO distribué et modèle d'acteur

Starzec *et al.* [2] proposent une version d'ACO où chaque fourmi est un acteur indépendant qui envoie des messages pour simuler les phéromones. L'idée est bonne, mais elle génère énormément de messages : chaque déplacement d'une fourmi en produit plusieurs. Avec Simovies, cela risque de surcharger les communications et de ralentir tout le système.

5.3 Programmation orientée agents

Cardoso et Ferrando [3] classent les agents du plus simple au plus complexe : d'un côté l'automate réactif (qui réagit juste à ce qu'il voit), de l'autre le modèle BDI où l'agent a des croyances, des désirs et des intentions. Notre implémentation se situe entre les deux : chaque robot réagit à ce qu'il perçoit, mais il peut aussi partager et recevoir des informations. C'est suffisamment simple et assez efficace pour le championnat.

5.4 Systèmes multi-agents et représentation des connaissances

Van der Hoek et Wooldridge [4] donnent une vue d'ensemble complète des SMA : comment les agents raisonnent, communiquent, et comment ils se coordonnent. C'est une bonne base théorique pour comprendre les concepts, mais elle reste très générale et ne donne pas d'indication sur comment implémenter ces idées dans un système temps-réel. Dans Simovies par exemple, les agents n'ont pas vraiment le temps de raisonner : ils doivent réagir immédiatement à chaque step. L'article est donc utile pour comprendre les concepts, mais pas directement applicable tel quel.

6 Présentation de l'Algorithme Implémenté

6.1 Organisation de l'équipe

L'équipe comprend 5 robots répartis en deux rôles :

- **3 robots MainBot** : ils patrouillent, détectent, communiquent *et tirent*
- **2 robots SecondaryBot** : ils patrouillent et détectent, mais ne tirent pas

6.2 Partage de la position ennemie

Quand un robot détecte un ennemi sur son radar, il obtient uniquement la distance et la direction vers cet ennemi. Pour que ses coéquipiers puissent les utiliser, il faut les convertir en coordonnées fixes sur la carte du terrain (x, y), compréhensibles par tous. Cette conversion est faite à l'aide de calculs trigonométriques, et le résultat est immédiatement envoyé à toute l'équipe via `broadcast()` au format "x,y". Ainsi, même un robot qui ne voit pas l'ennemi directement sait où il se trouve et peut se diriger vers lui.

```
1 if (object.getObjectType() == IRadarResult.Types.OpponentMainBot
2     || object.getObjectType() == IRadarResult.Types.OpponentSecondaryBot) {
3     double dist = object.getObjectDistance();
4     double direction = object.getObjectDirection();
5     Position enemyPosition = new Position(
6         position.x + dist * Math.cos(direction),
7         position.y + dist * Math.sin(direction));
8     broadcast(enemyPosition.toString());
9 }
```

Listing 1 – Calcul et envoi de la position ennemie

6.3 Réception des messages

Ce mécanisme est présent uniquement chez les robots MainBot : à chaque step, ils lisent tous les messages reçus depuis le step précédent via `fetchAllMessages()`. Chaque message contient les coordonnées (x,y) d'un ennemi détecté par un coéquipier, et le robot met à jour `enemyPosition` avec ces coordonnées pour pouvoir le poursuivre et tirer dessus.

```
1 ArrayList<String> messages = fetchAllMessages();
2 for (String message : messages) {
3     String[] enemyPos = message.split(",");
4     enemyPosition = new Position(
5         Double.valueOf(enemyPos[0]),
6         Double.valueOf(enemyPos[1]));
7 }
```

Listing 2 – Lecture et traitement des messages reçus

7 Tests de Performance

7.1 Méthode d'obtention des données de test

Pour chaque stratégie testée, on a lancé **20 parties** dans Simovies contre deux types d'adversaires : des robots Campfire et des robots CampBot. À chaque fin de partie, on a relevé le nombre de robots adverses immobilisés et le taux de survie des robots alliés. Ces valeurs ont ensuite été moyennées sur les 20 parties pour obtenir des résultats plus représentatifs et limiter l'impact des positions de départ aléatoires.

7.2 Résultats

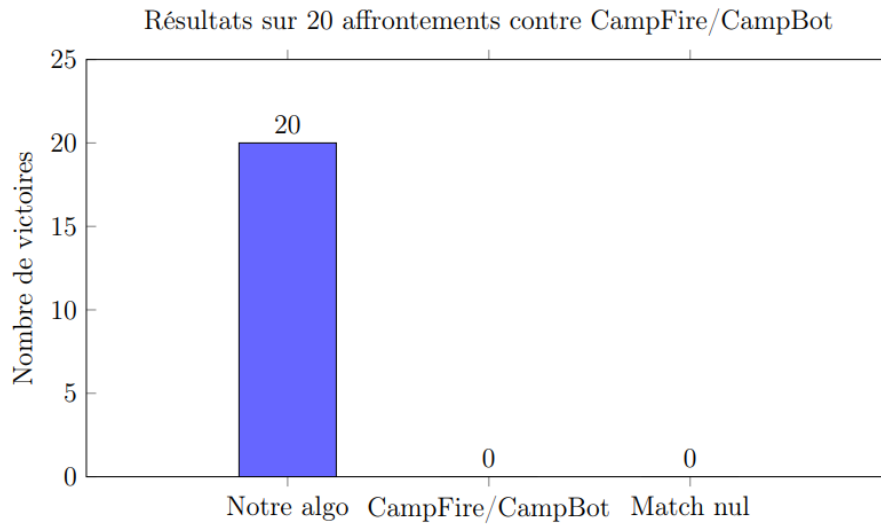


FIGURE 1 – Nombre de victoires de chaque équipe

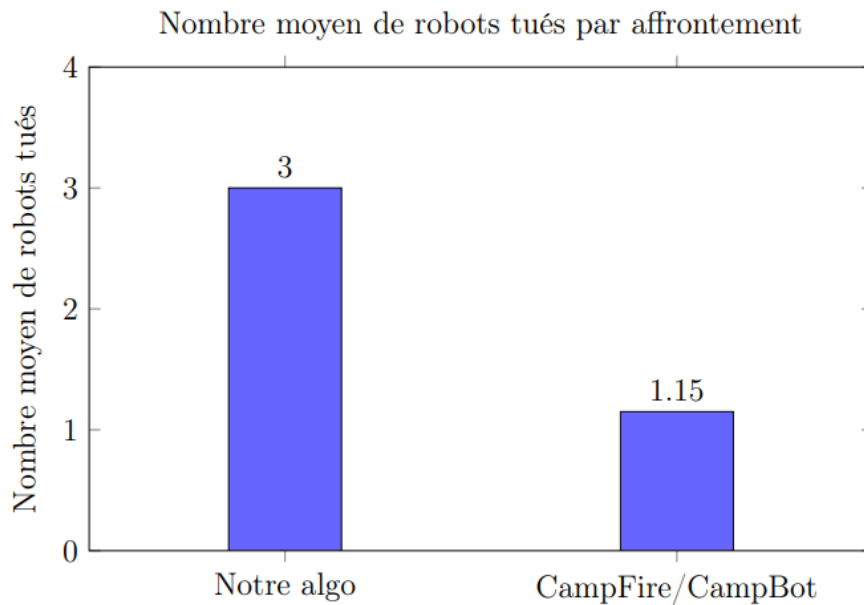


FIGURE 2 – Nombre moyen de robots tués de chaque équipe

7.3 Ce qui pourrait être amélioré

Plusieurs problèmes ont été identifiés :

- **Écrasement des messages** : Chaque nouveau message remplace le précédent. Si deux robots voient deux ennemis différents en même temps, seul le dernier message est gardé. Il faudrait gérer une liste de cibles.
- **Perte de cible trop rapide** : dès qu'un robot ne voit plus l'ennemi, il efface `enemyPosition`, même si un coéquipier le voit encore. Une meilleure gestion de la mémoire collective serait utile.

- **Les robots secondaires ne tirent pas** : ils ne font que détecter et relayer. Les activer en mode tir augmenterait la puissance de feu.

8 Conclusion

Ce projet nous a montré que le modèle d'acteur est une bonne façon d'organiser des robots autonomes : chaque robot fait ses propres choix, envoie et reçoit des messages.

Parmi les stratégies testées, la poursuite coopérative avec broadcast est celle qui fonctionne le mieux. Elle n'est pas parfaite, notamment parce qu'un seul message à la fois est géré et que tous les robots courent vers la même cible. Mais elle donne de bons résultats en championnat.

Références

- [1] A. Chakraborty and A.K. Kar, *Swarm Intelligence : A Review of Algorithms*, Nature-Inspired Computing and Optimization, Springer, 2017.
- [2] M. Starzec, G. Starzec, A. Byrski and W. Turek, *Distributed ant colony optimization based on actor model*, Journal of Computational Science, 2019.
- [3] R.C. Cardoso and A. Ferrando, *A Review of Agent-Based Programming for Multi-Agent Systems*, Electronics, MDPI, 2021.
- [4] W. van der Hoek and M. Wooldridge, *Multi-Agent Systems*, chapter 24 in Handbook of Knowledge Representation, Elsevier, 2007.